

第五章 白盒测试

一、单选题

1. 使用白盒测试方法时，确定测试数据的依据是指定的覆盖标准和（ ）？（单选题）

- A. 程序的注释
- B. 程序的内部逻辑
- C. 程序的需求说明
- D. 用户使用说明书

答案：B

解析：白盒测试（White-Box Testing）的核心特征是测试者能够看到并基于程序的内部结构和逻辑来设计测试用例。选项 B 正确，确定白盒测试数据的两个关键依据是：（1）指定的覆盖标准（如语句覆盖、分支覆盖、路径覆盖等），决定了测试的充分性目标；（2）程序的内部逻辑结构（控制流图、条件表达式、循环结构等），决定了测试数据如何设计才能达到覆盖目标。选项 A 错误，注释只是辅助理解代码，不能作为测试主要依据；选项 C 和 D 错误，需求说明书和用户手册是黑盒测试的数据来源。

2. 如果某测试用例集实现了某软件的路径覆盖，那么它一定同时实现了该软件的（ ）。（单选题）

- A. 判定/条件覆盖
- B. 组合覆盖
- C. 判定覆盖
- D. 条件覆盖

答案：C

解析：白盒测试的几种逻辑覆盖标准之间存在包含关系（由弱到强）：语句覆盖 $\subset$ 判定覆盖 $\subset$ 条件覆盖 $\subset$ 判定/条件覆盖 $\subset$ 条件组合覆盖 $\subset$ 路径覆盖。选项 C 正确，路径覆盖是最强的覆盖标准——它覆盖了程序中每一条可能的执行路径（从入口到出口的完整路径），因此自然覆盖了路径上的所有判定（即每个判定的每个分支都至少被执行一次）。但需注意路径覆盖不一定包含条件组合覆盖——路径覆盖覆盖了实际执行的路径上的条件组合，但可能遗漏某些理论上可能的条件组合。

3. 在下面所列举的逻辑测试覆盖中，测试覆盖最弱的是（ ）？（单选题）

- A. 条件组合覆盖
- B. 判定覆盖
- C. 语句覆盖
- D. 条件覆盖

答案：C

解析：白盒逻辑覆盖标准按强度从弱到强排列为：语句覆盖 $<$ 判定覆盖 $<$ 条件覆盖 $<$ 判定/条件覆盖 $<$ 条件组合覆盖 $<$ 路径覆盖。选项 C 正确，语句覆盖（Statement Coverage）是最弱的覆盖标准——它只要求每条语句至少被执行一次，但即使实现了 100% 语句覆盖，仍可能遗漏关键缺陷。例如代码中有 `if (a>0 && b>0)`，仅用 `a=1, b=1` 执行一次即可达到语句覆盖，但 `a=1, b=-1` 路径未被执行，`b>0` 为 false 时的逻辑错误无法被发现。因此在实际项目中，语句覆盖通常只作为最低门槛，更高的覆盖标准才能有效保证测试质量。

4. 一个程序中所含有的路径数与（ ）有着直接的关系。（单选题）

- A. 程序指令执行时间
- B. 程序语句行数
- C. 程序的复杂程度

D. 程序模块数

答案：C

解析：程序的路径数量主要由其控制流复杂度决定，而非代码行数。选项 C 正确，程序越复杂——分支嵌套层次越深、判断条件越多、循环结构越丰富——可能的执行路径数就越多，甚至呈指数级增长。圈复杂度（Cyclomatic Complexity）是衡量程序复杂度的常用指标，通过计算控制流图中独立路径的数量来评估。选项 A 和 B 错误，指令执行时间和代码行数与路径数没有必然关系——一段 100 行的顺序执行代码只有 1 条路径，而一段 20 行但包含多个嵌套 if-else 的代码可能有数十条路径；选项 D 错误，模块数量与单一模块内部路径数无关。

5. 白盒测试的测试数据是根据（ ）来设计的。（单选题）

A. 规格说明书

B. 结构

C. 内部逻辑

D. 应用范围

答案：C

解析：白盒测试设计测试数据的根本依据是程序的内部逻辑。选项 C 正确，测试人员通过分析源代码的控制流（if-else 分支、while/for 循环、switch 选择）、数据流（变量定义-使用的路径）和逻辑条件（复合布尔表达式），有针对性地设计测试数据以覆盖特定的逻辑结构和路径。选项 A 错误，规格说明书是黑盒测试的依据；选项 B“结构”表述不够精确——结构是内部逻辑的外在体现，内部逻辑才是本质。典型白盒测试设计步骤是：阅读源码→绘制控制流图→确定覆盖目标→设计测试数据→执行并验证覆盖率。

6. 某同学为此编写程序代码（Java）如下，假设已经传入参数并成功调用 process 方法。（简答题）

```
private static void process(double a, double b) {  
    double temp=0;  
    //a>0 条件和 b>0 条件的判断  
    if (a > 0 && b > 0) {  
        temp=a+b;  
        System.out.println("temp 的值为: ")  
    } else {  
        temp=a-b;  
        System.out.println("temp 的值为: ")  
    }  
}
```

请按照基本路径测试方法的以下步骤设计测试用例：

1. 依据代码绘制单条件流程图（注意上述代码是多条件判断），并转化为控制流图
2. 确定控制流图的圈复杂度
3. 确定线性独立路径的基本集合
4. 根据独立路径设计测试用例表覆盖每条基本路径，测试用例表按照指定 csv 格式填写

答案：

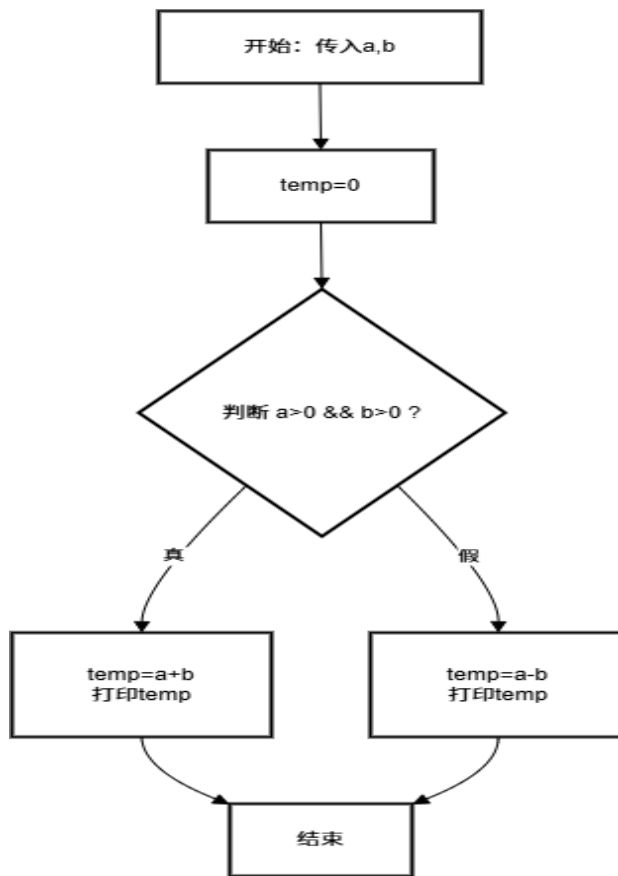
1. 原题梳理

核心分支: `if (a > 0 && b > 0)`

分支 1: `a>0 && b>0` 成立 $\rightarrow$ `temp=a+b`

分支 2: `a>0 && b>0` 不成立 $\rightarrow$ `temp=a-b`

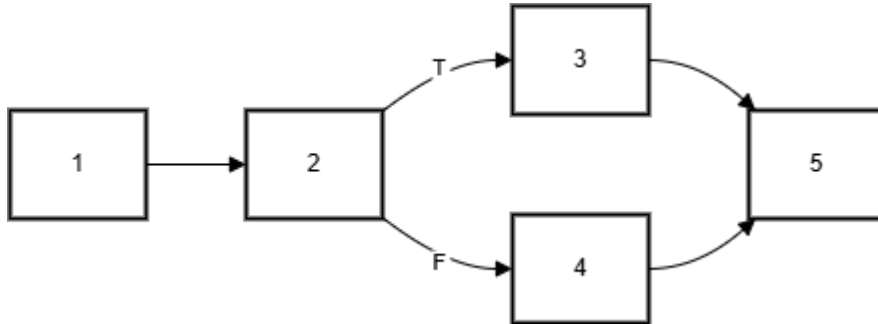
1) 流程图



2) 控制流图 CFG (Mermaid graph)

节点编号:

1. 入口、`temp=0`
2. 判断 `a>0 && b>0`
3. 真分支: `a+b` 输出
4. 假分支: `a-b` 输出
5. 出口



2. 计算圈复杂度 (3 种方法验证)

方法 1: 区域数 (包含外部区域)

控制流图区域:

区域 1: $2 \rightarrow 3 \rightarrow 5 \rightarrow 1 \rightarrow 2$

区域 2: $2 \rightarrow 4 \rightarrow 5 \rightarrow 1 \rightarrow 2$

区域 3: 外部区域

圈复杂度 $V(G) = 3$

方法 2: 判定节点 + 1

判定节点只有节点 2 (1 个判定)

$V(G) = \text{判定数} + 1 = 1 + 1 = 3$

方法 3: 边数 E - 节点数 N + 2

边 $E=5$, 节点 $N=5$

$V(G) = 5 - 5 + 2 = 3$

结论: 圈复杂度 = 3, 线性独立路径共 3 条

3. 线性独立路径基本集合

独立路径定义: 每条路径引入至少一条新边

路径 1: $1 \rightarrow 2 (T) \rightarrow 3 \rightarrow 5$

路径 2: 1 → 2 (F) → 4 → 5

路径 3: 1 → 2 → 3 → 5 ↔ 2 → 4 → 5 (闭合回路，补充独立环路)

简化标准基本路径集合（考试标准写法）：

- 1. 路径 1: 入口→判断为真→a+b→出口
- 2. 路径 2: 入口→判断为假→a-b→出口
- 3. 路径 3: 入口→真分支→出口，再回溯走假分支（环路独立路径）

简化应试版基本路径（最常用）：

Path1: 1-2 (T)-3-5

Path2: 1-2 (F)-4-5

Path3: 1-2 (T)-3-5-2 (F)-4-5

4. 测试用例表（CSV 格式）

设计思路

覆盖 3 条独立路径，构造 a、b 满足对应条件：

Path1: a>0 且 b>0

Path2: 不同时大于 0（3 种情况：a≤0；b≤0；都≤0）任选一组

Path3: 先满足 a>0&&b>0，再执行不满足分支

路径编号	a 取值	b 取值	判定条件 (a>0&&b>0)	预期 temp 结果	执行分支
Path1	5	3	true	8	a+b 分支
Path2	2	-4	false	6	a-b 分支
Path3	4, 再 - 1	6, 再 2	true→false	10, -3	先 a+b 再 a-b